



Data Types and Structures

Computational and Statistical Representations

Objectives

In this note, we will discuss:

- **computational** data *types*,
- NumPy **arrays**,
- **tabular** data,
- Pandas **data frames**,
- and **statistical** data *types*.

We will close with a discussion of the unfortunate lack of a clean mapping between computational and statistical data types, which will require persistent consideration throughout the course.

Python Setup

```
import numpy as np
import pandas as pd
```

Computational Data Types

Thinking about data types used in computing, there are three main categories:

- **Numeric** (Numbers)
- **String** (Text)
- **Boolean** (Logical)

Each of these types has (at least one) implementation in Python, NumPy, and Pandas.

Numeric

As the name suggests, numeric data is used to store numbers. Within numeric data, there are two broad subcategories: floats and integers.

Floats

Floating point numbers (floats) are a method of representing real numbers. In Python, some examples of floats include:

```
print(2.0)
print(42.0)
print(3.14)
```

2.0
42.0
3.14

We can verify that a Python scalar is a float using the `type` function.

```
type(1.1)
```

float

Real numbers, like π , can be irrational, and require a decimal representation with an infinite number of digits. Computers have finite memory, so of course they cannot store every digit of π , much less perform computations with infinite digits.

Without getting into the details of floating point arithmetic, we will simply note that floats will sometimes behave, oddly.

```
0.2 + 0.1 == 0.3
```

False

```
0.2 + 0.1
```

```
0.30000000000000004
```

This is normal, and in general will not cause issues.

There are technically different variants of floats, which use different numbers of bits to represent a number, however, we will generally not be concerned with this distinction.

Integers

Integers are whole numbers that can be positive, negative, or zero. Python represents integers using the `int` type.

```
print(2)
print(42)
print(-20)
```

```
2
42
-20
```

We can verify that a Python scalar is an integer using the `type` function.

```
type(-20)
```

```
int
```

Strings

Strings are sequences of characters that represent text data. In Python, you create strings by enclosing text in quotes, and Python represents them using the `str` type.

```
"Hello, World!"
```

```
'Hello, World!'
```

We can verify that a Python scalar is a string using the `type` function.

```
type("Hello, World!")
```

```
str
```

Boolean

Boolean data, otherwise known as logical data, stores the possible truth values `true` and `false`.¹ In Python, the two possible values are `True` and `False`.

```
print(True)
print(False)
```

True
False

We can verify that a Python scalar is a boolean using the `type` function.

```
type(True)
```

bool

Summary

The following code provides a few more examples to summarize computational data types in Python.

```
python_float = 3.14
python_int = 42
python_str = "hello"
python_bool = True
print(f"python_float is of type {python_float.__class__}")
print(f"python_int is of type {type(python_int)}")
print(f"python_str is of type {type(python_str)}")
print(f"python_bool is of type {type(python_bool)}")
```

```
python_float is of type <class 'float'>
python_int is of type <class 'int'>
python_str is of type <class 'str'>
python_bool is of type <class 'bool'>
```

NumPy Arrays

NumPy arrays are a fundamental data structure for scientific computing, and thus data science, in Python. Unlike Python's built-in data types like lists, NumPy arrays store elements of the *same type* efficiently and provide *vectorized operations*. We call this property of storing elements of the same type **homogeneous data**. They support the same basic data types we've discussed (floats, integers, strings, and booleans) but optimize them for numerical computation.

```
float_array = np.array([1.1, 2.2, 3.3])
int_array = np.array([1, 2, 3])
string_array = np.array(["a", "b", "c"])
bool_array = np.array([True, False, True])
print(f"Float array dtype: {float_array.dtype}")
print(f"Integer array dtype: {int_array.dtype}")
```

```
print(f"String array dtype: {string_array.dtype}")
print(f"Boolean array dtype: {bool_array.dtype}")
```

Float array dtype: float64

Integer array dtype: int64

String array dtype: <U1

Boolean array dtype: bool

For a comprehensive introduction to NumPy arrays and their capabilities, see the excellent [NumPy User Guide](#) . In particular, consider reading and working through:

- [NumPy: The Absolute Basics for Beginners](#) .

Tabular Data

In CS 307, the vast majority of the data that we encounter will be **tabular data**. If and when we encounter non-tabular data, we will usually try our best to transform non-tabular data to become tabular.

Generally speaking, tabular data refers to data that we can arrange in tables with **rows** and **columns**. This loose definition is useful, but there are extensions, such as [tidy data](#) , that further formalize this concept².

Thinking statistically:

- *rows* represent **observations**
- *columns* represent **attributes**

In machine learning, we will usually refer to the rows as **samples**. Columns will also be referred to as **variables**.

In both statistics and machine learning, some analyses will give additional categorization to the columns, usually denoting one column as the **target** or *response* and the remaining columns as the **features**.

x_1	x_2	x_3	y
1.2	yes	2	1
2.1	yes	3	0
5.5	no	6	0
4.3	no	4	1
2.7	yes	3	1
5.6	no	5	0
5.7	yes	7	1
9.8	no	8	0

x_1	x_2	x_3	y
3.6	yes	9	1
3.7	no	10	0

Table 1: An example of tabular data with ten samples (rows) and four variables (columns). By convention, y is used to denote the response, and the remaining variables x_1 , x_2 , and x_3 are called the features.

Pandas Data Frames

Pandas' `DataFrame` implements data frames in Python and are the primary data structure for tabular data analysis in Python. Unlike a NumPy array which stores homogeneous data, a `DataFrame` can store **heterogeneous data**. Specifically, within each column the data must have the same type, but different columns may store different types. You can effectively think of each column of a `DataFrame` as a NumPy array.³ This makes them ideal for real-world datasets that often contain a mix of numeric and categorical variables.

```
df = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Charlie'],
    'age': [25, 30, 35],
    'height': [5.5, 6.0, 5.8],
    'is_student': [True, False, False]
})

print(df)
print(f"")
print(df.dtypes)
```

```
   name  age  height  is_student
0  Alice   25    5.50         True
1   Bob   30    6.00        False
2 Charlie   35    5.80        False
```

```
name          object
age           int64
height        float64
is_student     bool
dtype: object
```

Note that strings in Pandas will often (but not always) have type `object`. Here we see 64-bit integers as well as 64-bit floats.⁴

For a comprehensive introduction to Pandas and its capabilities, see the excellent [Pandas User Guide](#). In particular, consider reading and working through:

- [10 Minutes to Pandas](#)

For a truly deep dive, see what is effectively the Pandas Book:

- [Python for Data Analysis](#)

Statistical Data Types

In statistics, data types have slightly different categorizations.

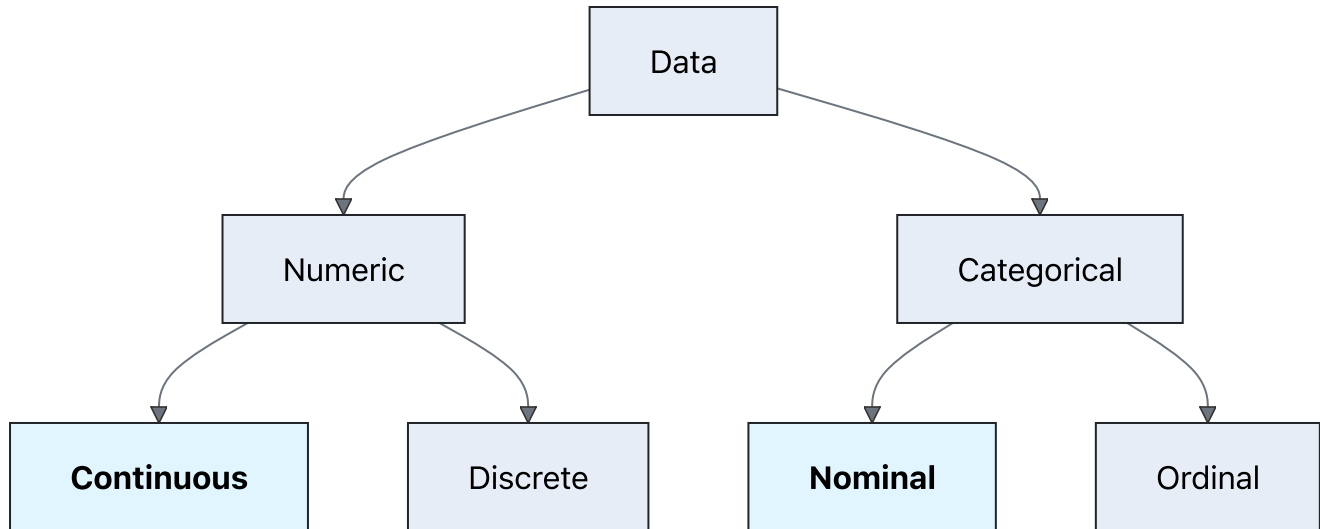


Figure 1: Flowchart displaying the hierarchy of statistical data types.

At the highest level, we classify data into two fundamental types. **Numeric data** consists of *measurements* that we express as numbers. **Categorical data** represents qualitative characteristics that classify observations into distinct groups or *categories*, where each observation belongs to one of a finite set of possible categories.

Numeric data can be further divided into two types. **Continuous** data consists of values that can take any real number in a range of values. Examples include height, weight, and temperature. These measurements can theoretically be made to any level of precision. That is, the only limit to precision (think: number of digits) is the precision of the measurement tool. **Discrete** data consists of values that are countable and in many cases are whole numbers. Examples include number of students in a class, the number of goals scored in a soccer match, and a person's age in years.

Categorical data can also be further divided into two types. **Nominal** data consists of categories with no inherent order or ranking. Examples include colors (red, blue, green), types of animals (cat or dog), or brands of cars (Honda, Toyota, Subaru). **Ordinal** data consists of categories with a natural order or ranking, but the differences between categories may not be equal. Examples include education levels (high school, bachelor's, master's, PhD), satisfaction ratings (poor, fair, good, excellent), or letter grades (A, B, C, D, F).

For simplicity, in CS 307 we will mostly focus on continuous numeric data, and nominal categorical data, essentially numbers and categories.⁵

Same Statistical Data, Four Representations for Computing

Suppose we had student data with an attribute (column) that denotes if they are a freshman, sophomore, junior, or senior. This is clearly **ordinal categorical** data from a statistical perspective as there is a natural ordering to class levels. However, we could represent this same information using different computational data types:

```
class_str = np.array(["freshman", "sophomore", "junior", "senior"])
print(f"Strings: {class_str}")
```

Strings: ['freshman' 'sophomore' 'junior' 'senior']

Using strings probably feels the most natural.

```
class_int = np.array([1, 2, 3, 4])
print(f"Integers: {class_int}")
```

Integers: [1 2 3 4]

We could just as easily use integers. Here, 1 represents a freshman, 2 a sophomore, and so on.

```
class_float = np.array([1.0, 2.0, 3.0, 4.0])
print(f"Floats: {class_float}")
```

Floats: [1. 2. 3. 4.]

Probably not the best idea, but floats would work as well.

```
is_fr = np.array([1, 0, 0, 0])
is_so = np.array([0, 1, 0, 0])
is_ju = np.array([0, 0, 1, 0])
is_se = np.array([0, 0, 0, 1])
print(f"One-hot encoded: {is_fr.dtype}")
```

One-hot encoded: int64

This last approach uses **one-hot encoding**, where we represent each category with a separate (integer) binary array containing a 1 when an observation is the category represented by the variable. Here, `is_fr` is 1 if the student is a freshman, `is_so` is 1 if the student is a sophomore, and so on. We'll discuss this representation technique in detail later in the course, as it's commonly used in machine learning.

string	integer	float	x_{freshman}	$x_{\text{sophomore}}$	x_{junior}	x_{senior}
freshman	1	1.0	1	0	0	0
sophomore	2	2.0	0	1	0	0
junior	3	3.0	0	0	1	0
senior	4	4.0	0	0	0	1

Table 2: Table displaying the same student class data in four computational representations: string, integer, float, and a one-hot encoding using four binary integer variables.

All four computational representations store the exact same **statistical information** - a student's class level. The choice of computational representation doesn't change the fact that this is ordinal categorical data, but it will affect how we need to handle the data in our analyses and models.

From Computational to Statistical Data

Unfortunately, as we saw in the last example, there is not a clean mapping from computing data types to statistical data types. We will of course store data using a computing data type. However, the statistical data types, which we will need to infer, will guide modeling decisions. In the previous example, we knew the statistical type, and thought about how to represent it numerically. However, in practice, we'll have data that is already stored for access in a computer, and instead we'll need to map a statistical type to a computational type.

Consider the following NumPy arrays:

```
a = np.array([1.0, 0.0, 1.0, 1.0, 0.0, 0.0, 0.0])
b = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 15, 25, 100])
c = np.array(["cat", "dog", "cat", "cat", "dog"])
d = np.array([True, True, True, False])
e = np.array([0.12, 0.35, 0.65, 0.89, 0.91])
```

These arrays have type:

```
print(a.dtype)
print(b.dtype)
print(c.dtype)
print(d.dtype)
print(e.dtype)
```

float64

int64

<U3

bool

float64

That is, we have at least one of each of the main computing types:

- Float: a
- Integer: b
- String: c
- Boolean: d
- Float: e

What is the statistical type of each? Are they numbers or categories? The answer is a common one you'll hear in CS 307: *it depends*.

In this case, without additional context, we would label them:

- a is categorical
- b is numeric
- c is categorical
- d is categorical
- e is numeric

How are we making this determination?

The general heuristics are:

- **Categorical** data represents a *finite* number of *categories*.
- **Numeric** data represents *measurements* or *counts*.

Why is a categorical? Well, technically, with the information we have here, we don't know for sure. However, notice that there are only two values contained in the array, 0.0 and 1.0. While it is certainly reasonable to think that floats are (continuous) numeric data, that is not necessarily the case.

- The a variable could represent a binary indicator, something like 1.0 for graduate students and 0.0 for undergraduate students.
- The a variables could instead represent a measure, something like 1.0 millimeters of precipitation and 0.0 millimeters of precipitation, and by chance we only observed two possible values.

Which situation are you dealing with? You'll have to pay close attention to the data you're working with! In theory, when you're working with high quality data, you will have access to a **data dictionary** that describes each variable so you can better determine how to use it in a statistical model.

Footnotes

1. Some languages, like R, provide for a third possibility that effectively represents an unknown status. ↩
2. [Database normalization](#) is a process that is deeply invested in how data is stored as rows and columns within a relational database. ↩
3. They are of course actually a Pandas *Series*, but importantly, they are a one-dimensional collection of data of the same type. ↩

4. But again, we won't worry much about this. For us, if they were 32-bit floats, it wouldn't change our analysis. ↩

5. Dealing with discrete numeric and ordinal categories requires care that we may return to later with specific examples. ↩

This website is developed and maintained by David Dalpiaz for CS 307 at UIUC .